BELLMAN-FORD ALGORITHM ● SIDE 1: CODE

```
#include <iostream>
using namespace std;

const int MAX = 100;
const int INF = 99999999;

struct Edge {
    int u, v, weight;
};
void inputEdges(Edge edges[], int n, int e) {
    cout << "Enter edges (Source Destination Weight):\n";
    cout << "(Negative weights are allowed)\n";
    for (int i = 0; i < e; i++) {
        int u, v, w;
        cin >> u >> v >> w;
        if (u >= 1 && u <= n && v >= 1 && v <= n && u != v) {
            edges[i].u = u - 1;
            edges[i].v = v - 1;
            edges[i].weight = w;
        } else {
            cout << "Invalid edge. Try again.\n";
            i--;
        }
    }
}

// Bellman-Ford Algorithm - relaxes all edges (n-1) times
void bellmanFord(Edge edges[], int n, int e, int source) {
    int dist[MAX];
    for (int i = 0; i < n; i++) dist[i] = INF;
    dist[source] = 0;

    // relax all edges (n-1) times
    for (int iteration = 1; iteration <= n - 1; iteration++) {
        bool updated = false;
        cout << "\n--- Iteration " << iteration << " ---\n";
        for (int i = 0; i < e; i++) {
            int u = edges[i].u, v = edges[i].v, w = edges[i].weight;
            if (dist[u] != INF && dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                cout << "Edge " << u+1 << " -> " << v+1
                     << " updated. dist[" << v+1 << "] = " << dist[v] << "\n";
                updated = true;
            }
        }
        if (!updated) {
            cout << "No updates. Distances stabilized early.\n";
            break;
        }
    }
```

```
    // extra pass to detect negative weight cycles
    bool hasNegCycle = false;
    for (int i = 0; i < e; i++) {
        int u = edges[i].u, v = edges[i].v, w = edges[i].weight;
        if (dist[u] != INF && dist[u] + w < dist[v]) {
            hasNegCycle = true;
            break;
        }
    }
    if (hasNegCycle) {
        cout << "\nGraph contains a negative weight cycle!\n";
        return;
    }
    cout << "\nShortest Distance from Node " << source+1 << ":\n";
    for (int i = 0; i < n; i++) {
        if (dist[i] == INF)
            cout << source+1 << " -> " << i+1 << " = INF\n";
        else
            cout << source+1 << " -> " << i+1 << " = " << dist[i] << "\n";
    }
}
int main() {
    int n, e;
    const int MAXE = MAX * MAX;
    Edge edges[MAXE];
    cout << "Enter number of nodes: ";
    cin >> n;
    cout << "Enter number of edges: ";
    cin >> e;
    inputEdges(edges, n, e);
    int source;
    cout << "\nEnter source node: ";
    cin >> source;
    bellmanFord(edges, n, e, source - 1);
    return 0;
}
```

BELLMAN-FORD ALGORITHM ● SIDE 2: WHEN, HOW & GOTCHAS

WHEN TO USE BELLMAN-FORD (Scenario Triggers)

- Graph has NEGATIVE edge weights — Dijkstra's cannot be trusted here
- Question asks to DETECT a negative weight cycle (e.g., arbitrage detection, currency exchange loops)
- Question mentions "iterations", "relaxation", or gives an iteration-by-iteration table to fill
- Single-source shortest path needed, but negative edges may exist
- "Some routes give a refund/bonus (negative cost)" style scenarios

Bellman-Ford vs Dijkstra's — Which One?

- Negative edges present? → Bellman-Ford (Dijkstra's gives wrong answers)
- All weights non-negative? → Dijkstra's is faster, prefer it
- Need to DETECT a negative cycle specifically? → Bellman-Ford is the only option here
- Bellman-Ford is slower ($O(V \times E)$) but more general than Dijkstra's ($O(V^2)$ or better)

MANUAL TRACE METHOD (Iteration Table, from Class Notes)

Use this exact process — matches the class notes' iteration-column table format:

- 1. Make a table: one column per iteration (1st, 2nd, 3rd... up to n-1 iterations).
- 2. Set dist[source]=0, all other nodes = INF (α), BEFORE iteration 1.
- 3. In each iteration, go through EVERY edge (in given order) exactly once.
- 4. For edge (u,v,w): if dist[u] + w < dist[v], UPDATE dist[v] and write the new value in that column.
- 5. If an edge does NOT cause an update, write 'no update' in that column (matches class notes style).
- 6. Repeat for (n-1) total iterations, OR stop early if a full pass causes zero updates.
- 7. Run ONE more pass after the last iteration — if ANY edge can still relax, a negative cycle exists.

COMMON GOTCHAS (Exam Traps)

⚠ Watch out for:

Exactly (n-1) iterations are needed in the worst case — this guarantees correctness for graphs with no negative cycle. Stopping too early may miss valid updates.

The EXTRA (n-th) pass is what DETECTS a negative cycle — if any edge can still relax after (n-1) iterations, a negative cycle exists and shortest paths are undefined.

Edge order matters for the TRACE (which values update in which iteration) but NOT for the final answer — final shortest distances are the same regardless of edge order.

Unlike Dijkstra's, Bellman-Ford does NOT mark nodes 'visited' — it simply relaxes ALL edges repeatedly, so it can correctly handle a later negative edge improving an earlier decision.

dist[u] must NOT be INF when relaxing edge (u,v) — check dist[u] != INF first, otherwise INF + weight causes overflow/garbage values.

BELLMAN-FORD ALGORITHM ● SIDE 3: WORKED EXAMPLES

EXAMPLE 1 — Small Graph with Negative Edge (Trace by Hand)

Graph (directed): S->T=6, S->Y=7, T->X=5, T->Y=8, T->Z=-4, X->T=-2, Y->X=-3, Y->Z=9, Z->S=2, Z->X=7

Source: S

Step-by-step (simplified trace):

- Init: dist[S]=0, all others INF
- Iteration 1: dist[T]=6, dist[Y]=7 (from S). Then dist[X]=min(INF,6+5)=11, dist[Z]=min(INF,6-4)=2
- Iteration 2: dist[X]=min(11,7-3)=4 (via Y better). dist[Z] stays 2 (no improvement yet)
- Iteration 3: dist[T]=min(6,4-2)=2 (via X better!). This makes dist[Z]=min(2,2-4)=-2 (via improved T)
- Iteration 4: stabilizes, no more updates → stop early

Final shortest distances from S: S=0, T=2, X=4, Y=7, Z=-2 (matches CLRS textbook example, verified against compiled code)

EXAMPLE 2 — Negative Cycle Detection

Graph: A->B=1, B->C=-3, C->A=1 (cycle: A->B->C->A has total weight 1-3+1 = -1, a NEGATIVE cycle)

After (n-1)=2 iterations, run ONE more pass: edge B->C can still relax dist[C] further.

Exam expects you to say: "Since an edge can still relax after n-1 iterations, this graph contains a negative weight cycle. Shortest paths through this cycle are undefined (can be made arbitrarily small by looping)."

EXAMPLE 3 — Real-World Scenario (CP-style)

Scenario: "A currency trader wants to know if there's a sequence of currency exchanges that results in a profit through arbitrage (exploiting exchange rate differences)."

How to apply Bellman-Ford: currencies = nodes, exchange rates = edges. Convert exchange rates to weights using -log(rate) so that a PROFITABLE arbitrage loop becomes a NEGATIVE weight cycle. Running Bellman-Ford's negative-cycle-detection step directly answers whether profitable arbitrage exists.

EXAMPLE 4 — Why Not Dijkstra's? (Medium/Hard style)

Scenario: exam gives a graph with one negative edge and asks "Can we use Dijkstra's here? Justify your answer."

Correct handling: "No. Dijkstra's greedily finalizes a node's distance once visited, assuming no future edge can improve it. A negative edge discovered later could still reduce that 'finalized' distance, but Dijkstra's would never revisit it — producing an incorrect result. Bellman-Ford avoids this by relaxing ALL edges repeatedly across multiple passes, allowing later negative edges to still correct earlier distances."

QUICK CHECKLIST BEFORE SUBMITTING AN ANSWER

Before you finalize any Bellman-Ford answer, check:

Did I run exactly (n-1) iterations (or stop early only if a full pass had zero updates)?

Did I run the EXTRA pass to check for a negative cycle?

Did I check dist[u] != INF before relaxing an edge from u?

If a negative cycle exists, did I state shortest paths are undefined (not just report wrong numbers)?

Did I show the iteration-by-iteration table, not just the final answer?